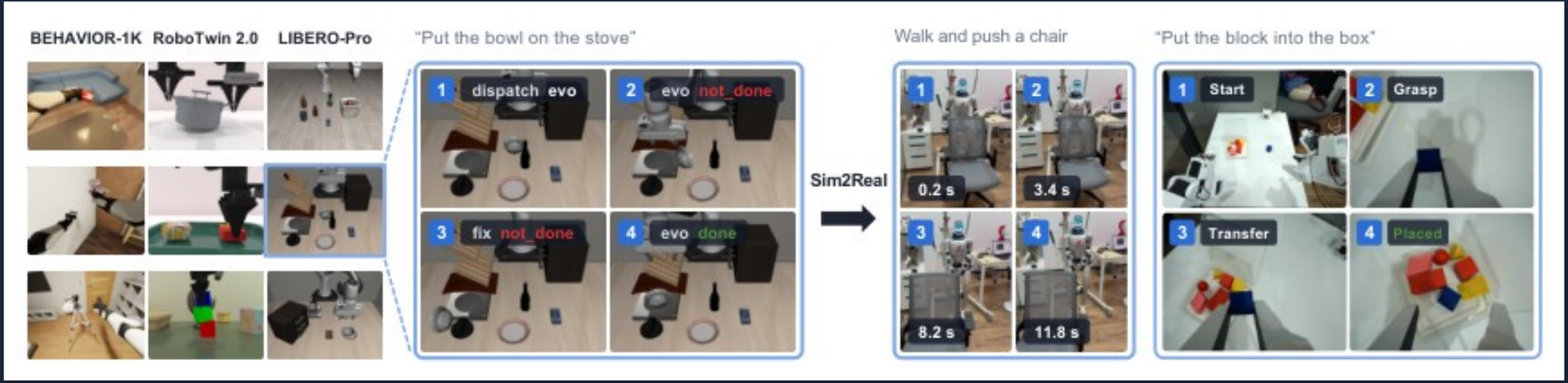
REA: No Waiting on the Brain

REA: A Routed Embodied Agent Runtime for Asynchronous Brain-Cerebellum Control

Pengfei Zhou, Yixing Ma, Zhengfeiyang Zhang, Haoyuan Chen, Zhenglin Wan, Hexin Wang, Wangbo Zhao, Yang You
NUS HPC-AI Lab · InfRec · UC Berkeley · U Tokyo · SIAT CAS · HKUST



Paper review | arXiv 2610.42053 (2026-10-30) | Figure: paper Fig. 2

One-sentence summary

Offline brain, boundary judging, table-lookup dispatch

REA: three-layer embodied runtime of brain (System 2), router (System 1), cerebellum (System 0): brain builds policies offline, judges completion only on return; router dispatches by measured dev-set success, escalating recovery in a shared budget — **running policies never wait for the brain.**

95.9%

LIBERO-Pro success
Best single method 94.7%

−17%

Control time vs Agents SDK
−25% vs blocking advisor

86%

RoboTwin VLA + code
Best single VLA 80%

8/10

Real G1 chair pushing
Best baseline 5/10

Contribution: systems and evaluation — a unified three-layer contract + three router modules (route memory / judge gate / escalation ladder) + 30,000+ aligned test episodes.

Background & motivation

Slow, smart brain; fast, brittle cerebellum; no way to combine them

System 2 brain: strong reasoner, slow and costly

43 min

ASPIRE repair loops per LIBERO task (median)

89M

Codex + GPT-6 input tokens to program 60 tasks

1 / 1,987

Token-by-token action VLMs: successes on LIBERO-Pro

0 / 600

3 cm / 15° Cartesian IK: direct-control successes

System 0 cerebellum: fast, poor generalization

VLAs / whole-body controllers

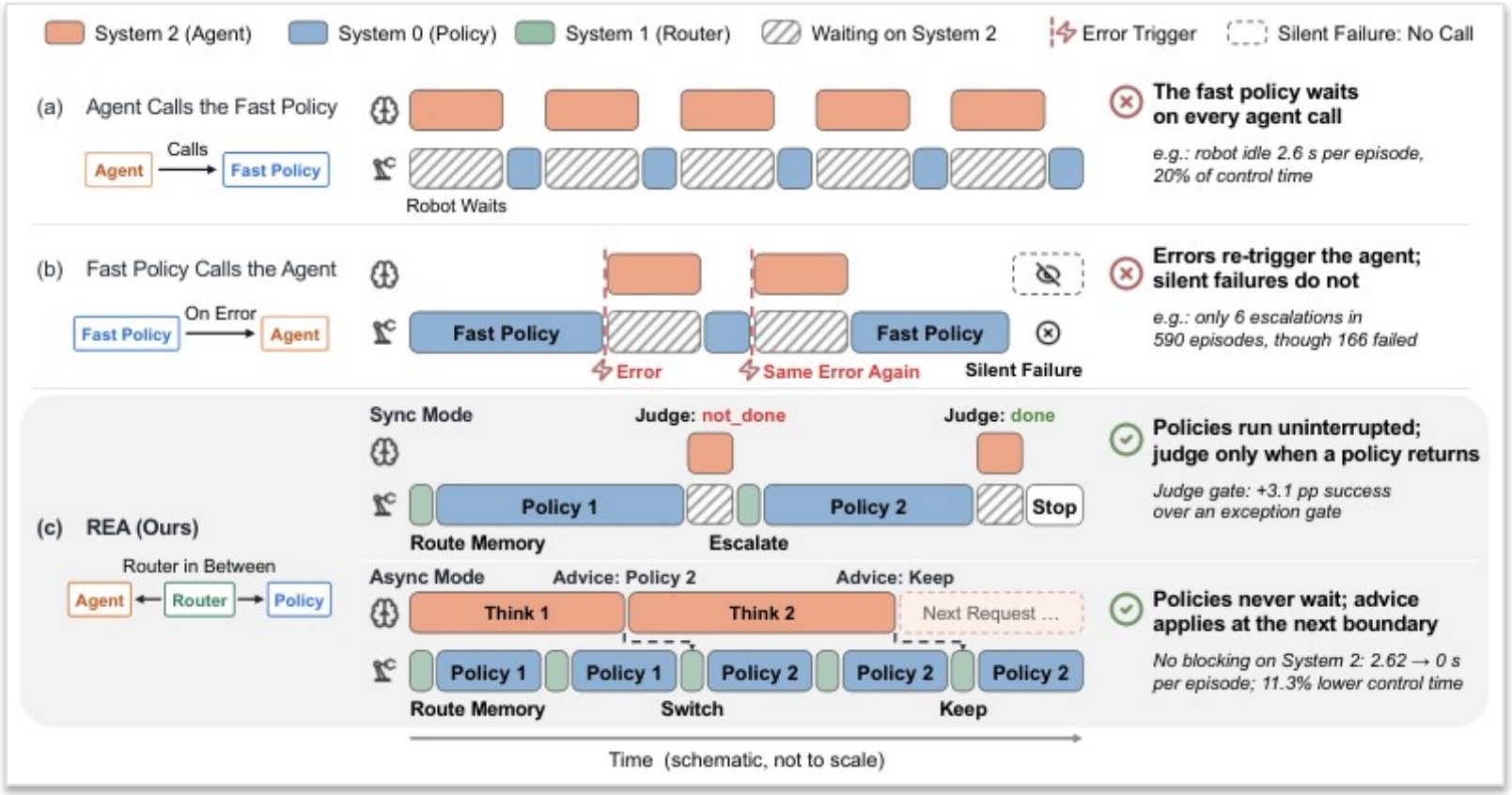
- Runs at a few to tens of Hz, far faster than agent reasoning
- VLAs scoring >90% on LIBERO collapse on perturbed LIBERO-Pro
- Good as "reflexes", not for handling OOD tasks alone

Obstacle 1 No shared runtime: agents and VLAs use different simulators, observation formats and success criteria, so they cannot be compared on the same episodes.

Obstacle 2 Hard async coordination: a brain calling the cerebellum stalls execution; a cerebellum calling the brain on errors triggers repeatedly yet misses failures that raise no error.

Core problem

Common couplings fail: robots wait, or silent failures go unseen



(a) Agent calls fast policy

Robot waits every time: 2.6 s idle / episode, 20% of control time

(b) Policy calls agent on error

166 of 590 ep fail, yet exceptions trigger only 6 escalations — most failures are "silent"

(c) REA

Lookup dispatch, policies run at native rate; judge done / not_done only at policy return

Method: three-layer contract

Brain advises, router manages control flow, policies just execute

System 2 • Brain

Coding agent / VLM

Offline Generate(t) creates candidate policies; online Judge(f_0, f_k, ℓ) → done / not_done, Advise(o) recommends a candidate

System 1 • Router

Outputs no actions

Decides which policy runs, when to consult the brain and when to stop; never blocks a running policy

System 0 • Cerebellum

Code, VLA, whole-body control

Runs closed-loop at native rate; returns control to the router at segment or policy boundaries

Unified execution interface

$$(\Delta, \delta, e, \tau) \leftarrow \text{Run}(p; H-u, W-w)$$

Receives only remaining step/wall-clock budget; returns usage, exception e, truncation flag τ

Three experimental variants

REA-M	route memory picks policy + exception gate; zero online brain calls
REA-A	Agent handles initial dispatch and post-exception recovery choice
REA-V	route memory + visual judge at every policy return

Method: three router modules

One module per coordination problem: dispatch, judging, recovery

① Route memory

Dispatch bypasses the brain

- ~10 runs per candidate on dev seeds (disjoint from test)
- Record per-task success rate $\hat{r}$ and mean steps $\hat{c}$
- At test time, argmax $\hat{r}$; ties go to fewer steps
- Dispatch = one table lookup, 0 tokens

② Judge gate

Catch silent failures on return

- One visual Judge call per Run return
- done → stop, protecting completed work
- not_done → escalate to next rung
- Baseline: free exception gate, escalates only on exception/truncation

③ Escalation ladder

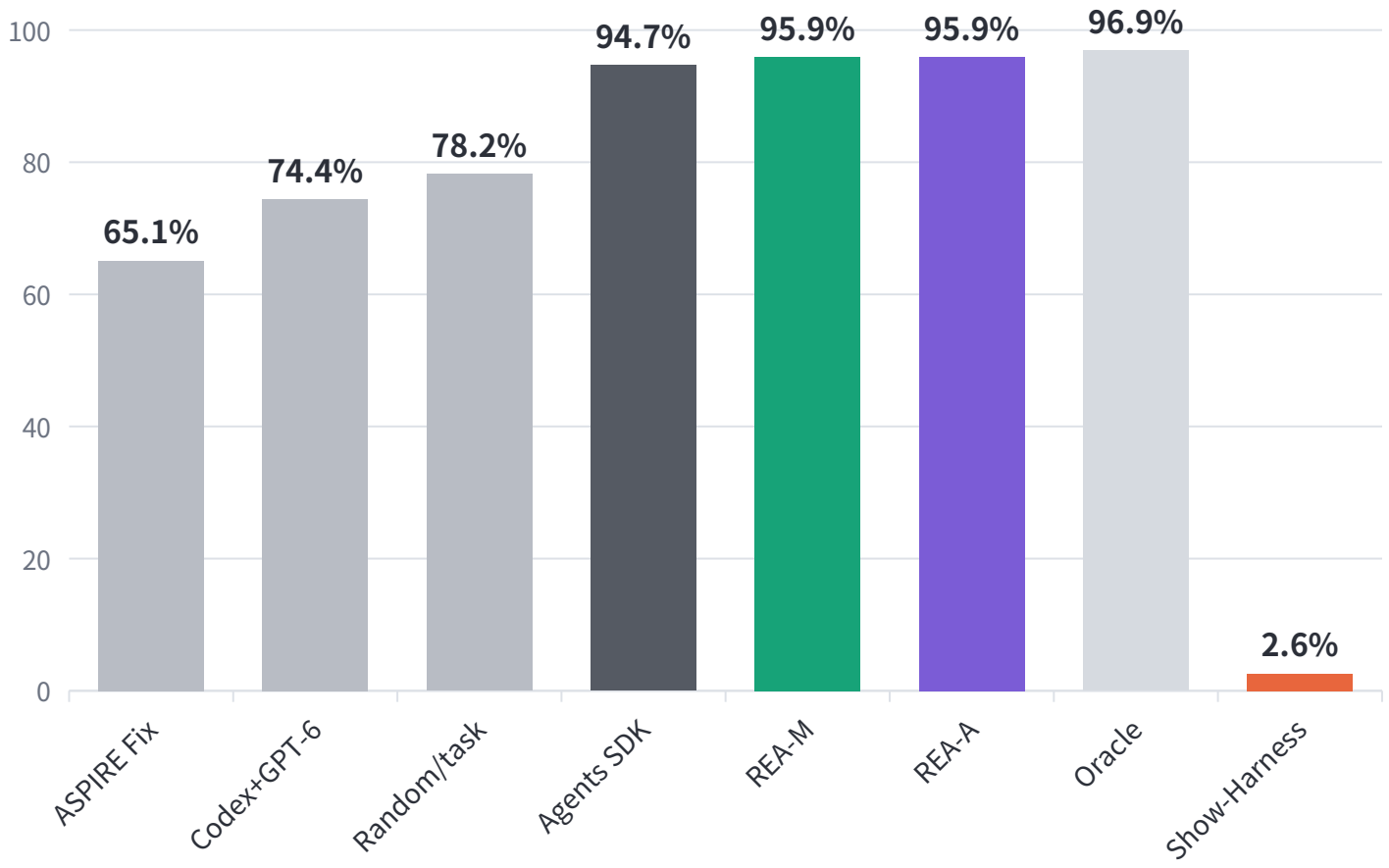
Capped recoveries and budget

- Ladder L = [first, other, first]
- Each rung continues from the current state
- All runs and brain calls share (H−u, W−w)
- An episode never exceeds the single-policy budget

Example (Table 12): on GS-drawer, dev-set Fix 1/10, Evo 10/10 → dispatch Evo first; ladder [Evo, Fix, Evo].

Routing, not pool size: 95.9% beats every single pooled method

Success rate (%), all 60 tasks



+1.2 pp

vs best single method Agents SDK (86:51, p=0.004)

95.3 vs 93.7

37 strictly aligned tasks (same code, p=0.001)

1,003 vs 1,343

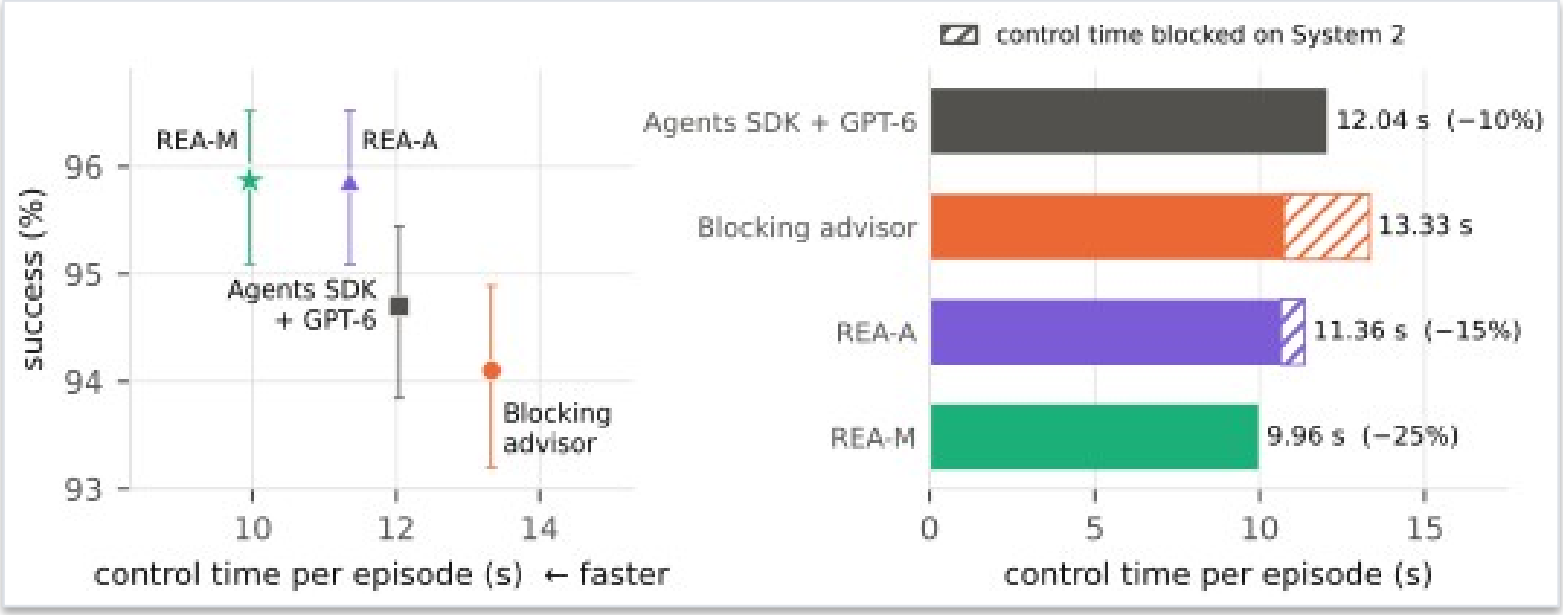
Control steps per success (−25%)

0 token

route memory uses only 1,790 dev episodes; generation pipelines use 29–89M tokens each

Efficiency

Brain out of loop: REA-M fastest timed method, 25% less control time



Of 1,236 episodes where the blocking advisor overrode route memory, route memory's policy solves 1,168; the advisor only 1,124.

2.62 s

Blocking-advisor idle time per episode (20% of control time)

9.96 s

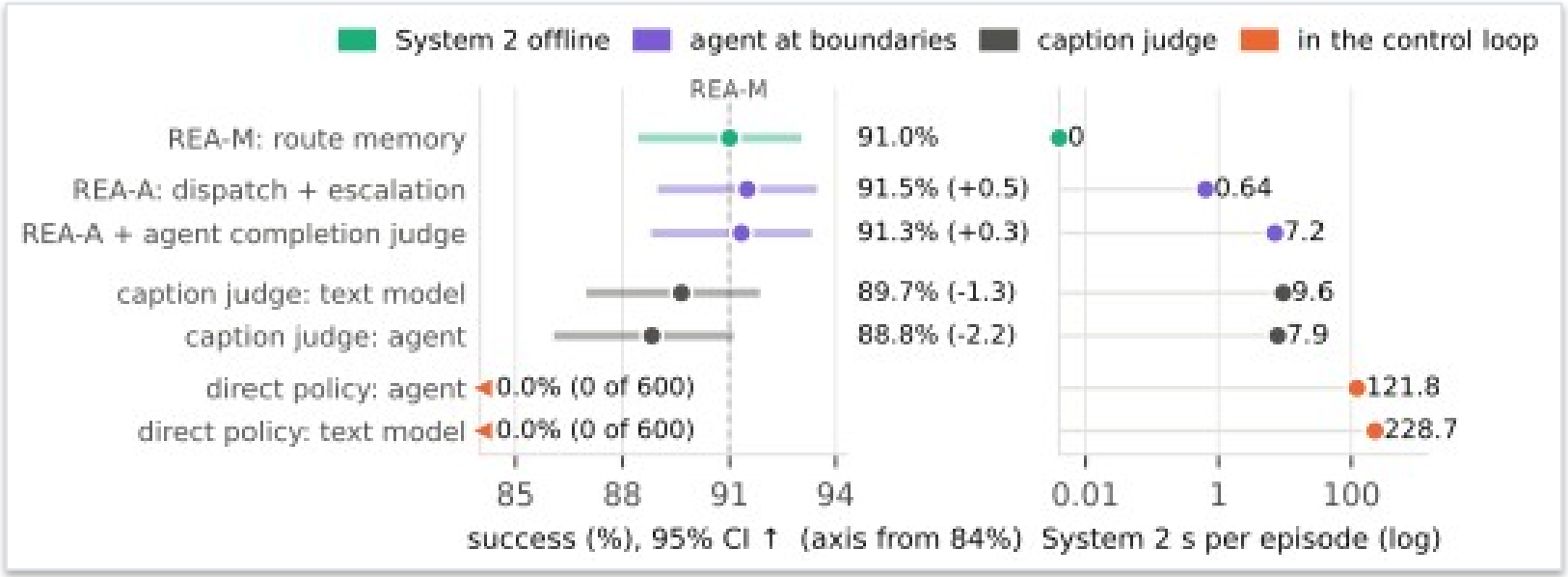
REA-M control time per episode (−25% vs advisor, −17% vs SDK)

10.3 ms

Same per-step cost for both → gap comes from waiting and step count, not hardware

Where to put System 2

Brain works best offline; in the control loop, success hits zero

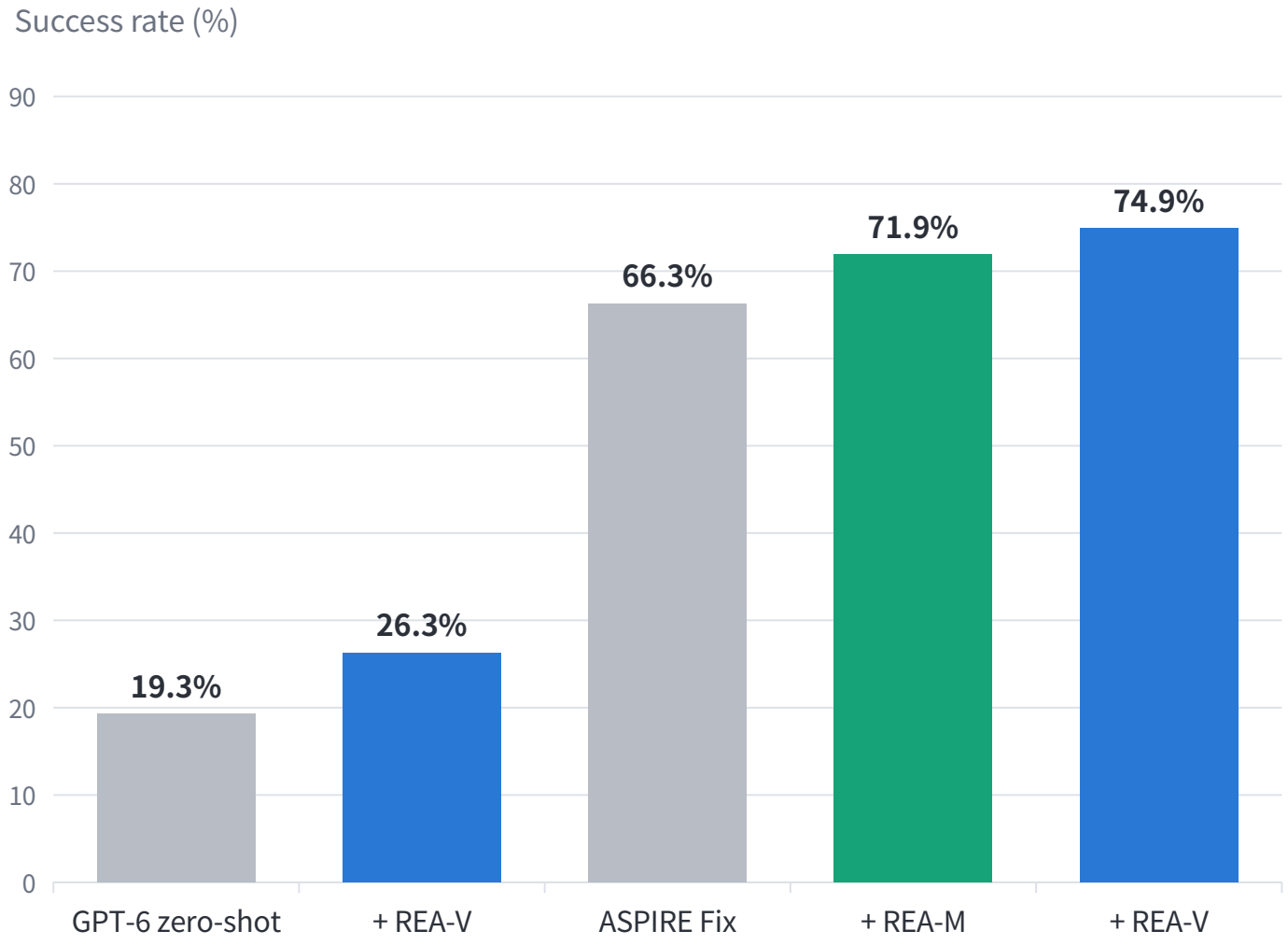


Placement	Success / brain time
Offline (REA-M)	91.0%, 0 s
Dispatch + recovery (REA-A)	91.5%, 0.64 s
+ agent completion judging	91.3%, 7.2 s
Text-description judging	88.8–89.7%, 7.9–9.6 s
Direct control policy	0.0%, 122–229 s

Conclusion: the brain adds most value generating policies offline; online, it is worth paying for only as a visual judge when policies fail silently.

Across policy libraries

REA lifts policy libraries, whether strong or weak



ASPIRE library: 66.3% → 74.9%

- REA-M +5.6 pp (49:16), while steps/success drop: 1,557 → 1,467
- Exception gate fires only 6 times → gains come almost entirely from route memory dispatching Evo first
- judge gate adds another +3.1 pp (p=0.022)

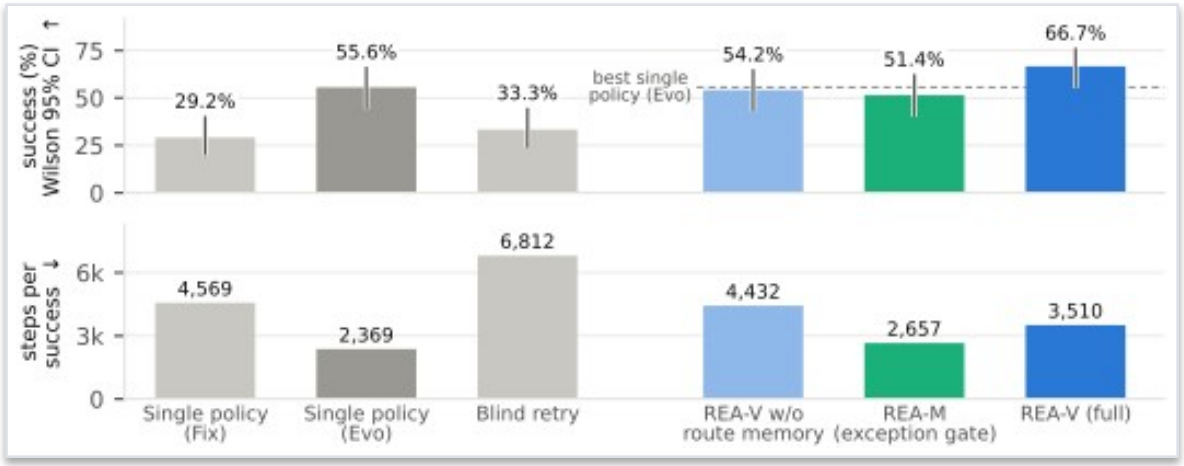
Weak library: 19.3% → 26.3%

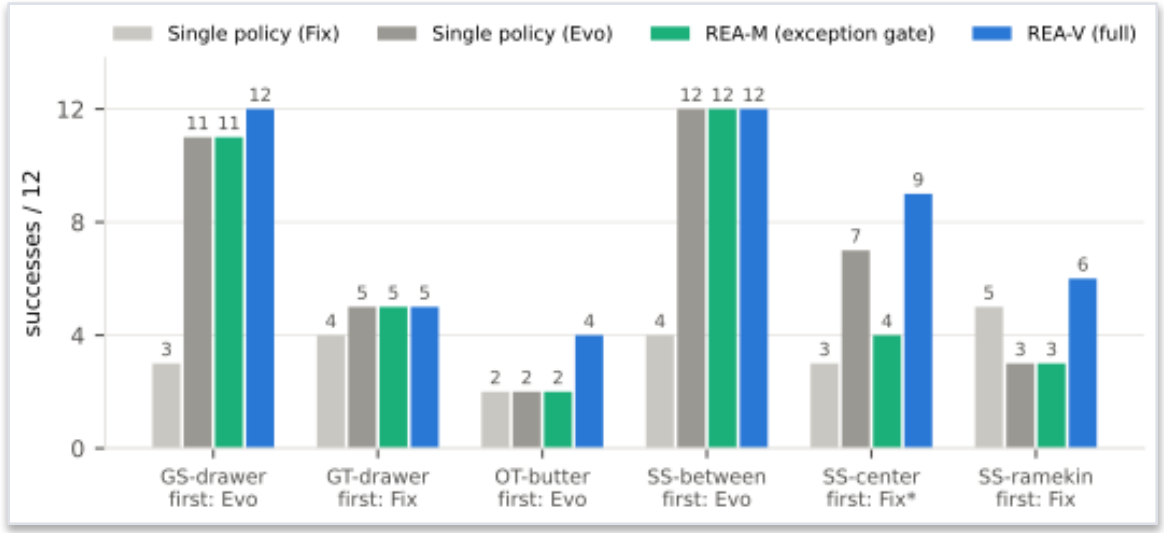
GPT-6 zero-shot: routing between two samples + judge (50:8, $p<10^{-4}$)

Cost: the judge gate brings ~2.1× wall-clock time per success (40.9 → 120.7 s).

Ablation • hard subset

Both modules needed: only the full router tops the best single policy





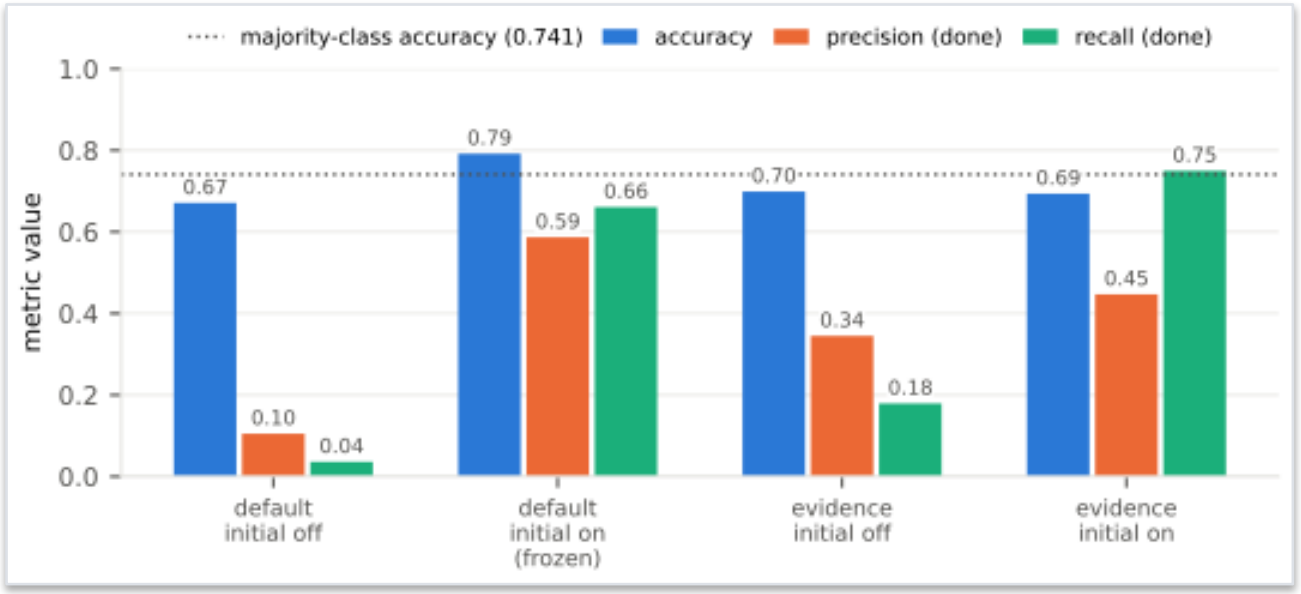
66.7%
Full REA-V (48/72); best single policy Evo 55.6%

−15.3 pp
judge gate → exception gate (12:1, p=0.003)

33.3%
Uncoordinated reruns barely help

Mechanism

Why a noisy judge helps: precise "done" calls, reruns rarely hurt



Calibration (216 dev labels): with the initial frame, accuracy rises 0.67 → 0.79, the only config above the 0.741 majority-class baseline; stable for thresholds 0.5–0.9.

0.83 / 0.905

done precision (hard subset / LIBERO-Pro)

24% / 19%

Failed first runs rescued by rerun

5% / 9%

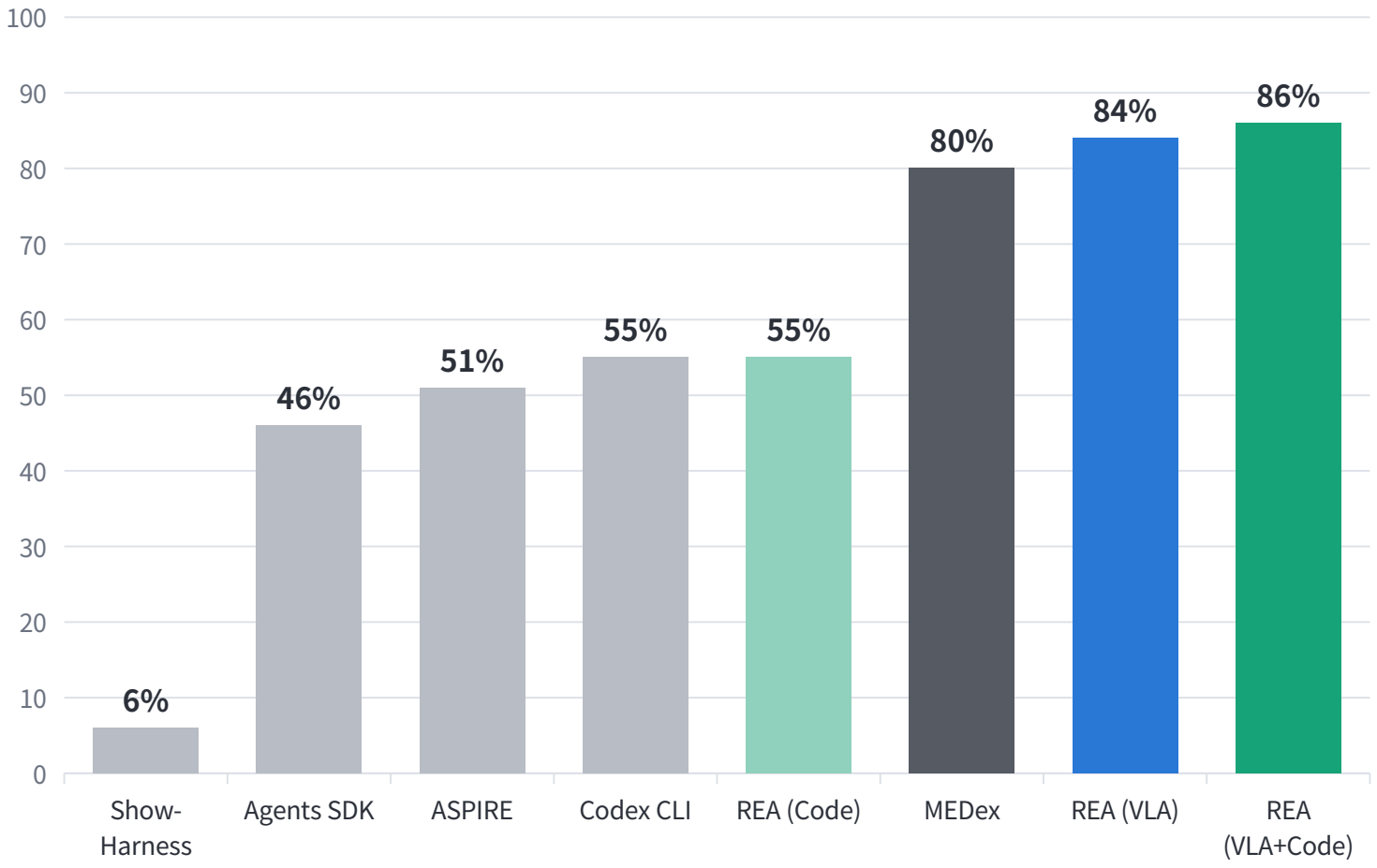
Successful first runs broken by rerun

Note: overall final-judgment accuracy is only 0.72 / 0.67; the authors call this "a consistent explanation, not a tested mechanism."

VLA + code

VLA + code, one interface: mixed pool's 86% beats best single VLA

Overall success rate (%), 100 scenes



Learned System 1 selector

- Qwen3.5-4B + LoRA reads initial instruction, three-view images and robot state
- One sigmoid success head per candidate; pick the top one at start, fixed for the episode

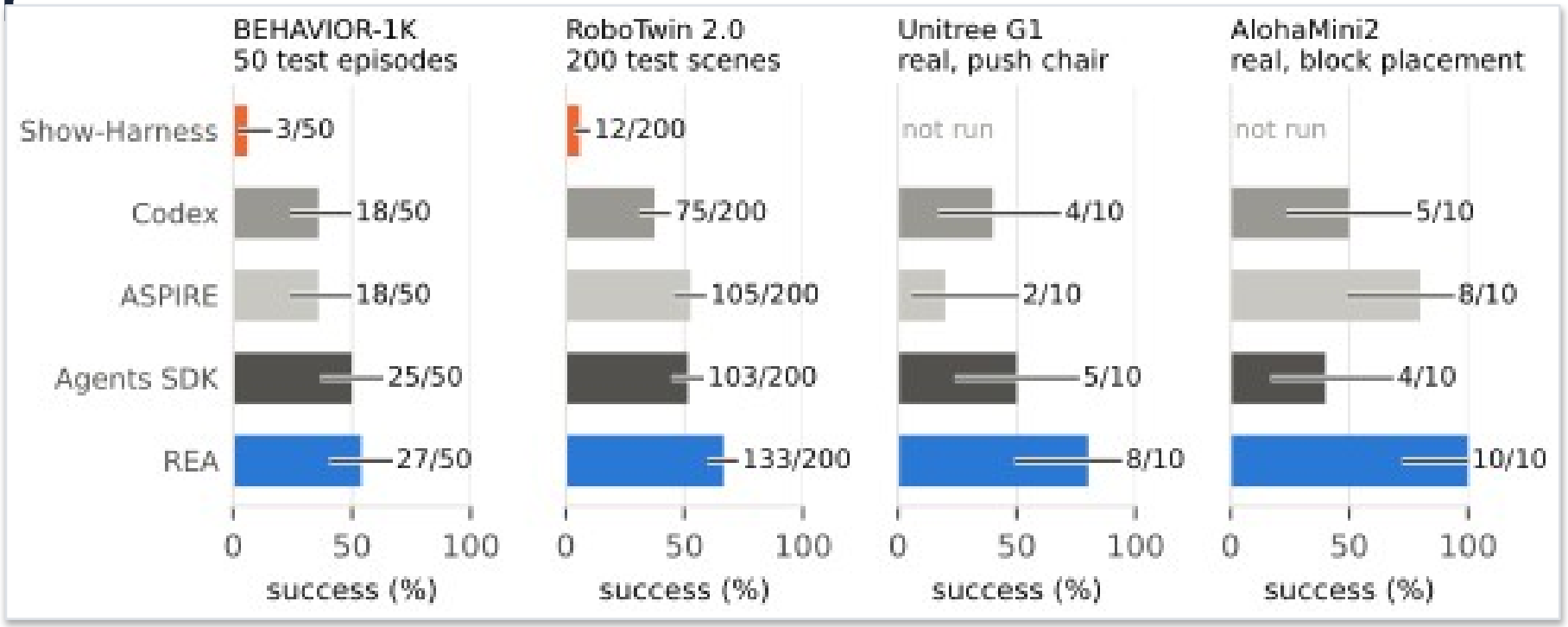
Contribution of code policies

Code chosen in 9 randomized scenes, 8 succeed; net +2, no losses

Caveat: baselines are historical runs from different time windows, MEDex on another host; no mid-episode switching or judge gate here.

Transfer

Swappable backends: REA tops observed successes in every transfer



BEHAVIOR-1K

27/50 vs SDK 25/50

Gap not significant (7:5, p=0.77), mainly from the single camera task

RoboTwin 2.0 • 200 scenes

97 vs Codex 75 (same night)

REA 133 on another night; same config differs by 36 scenes across nights → window effect

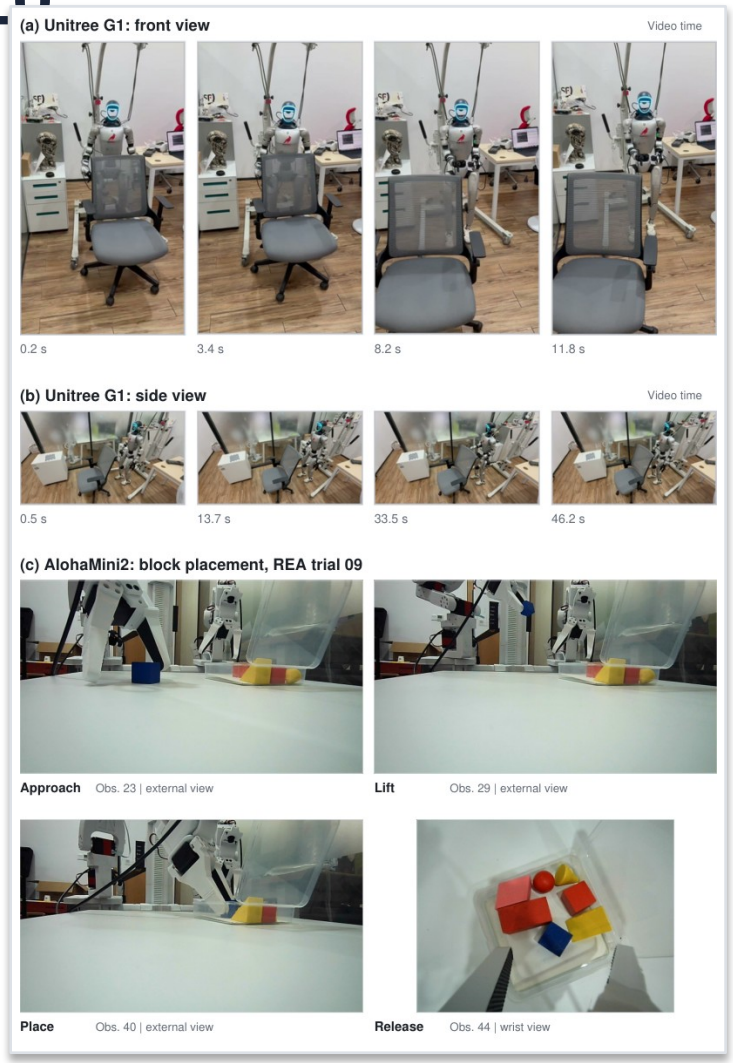
Real robot

G1 8/10 • Aloha 10/10

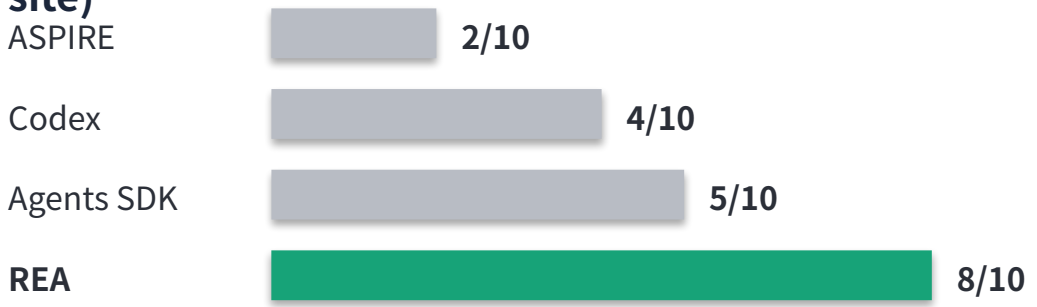
10 trials per method, human-judged; REA slower on Aloha

Real robot

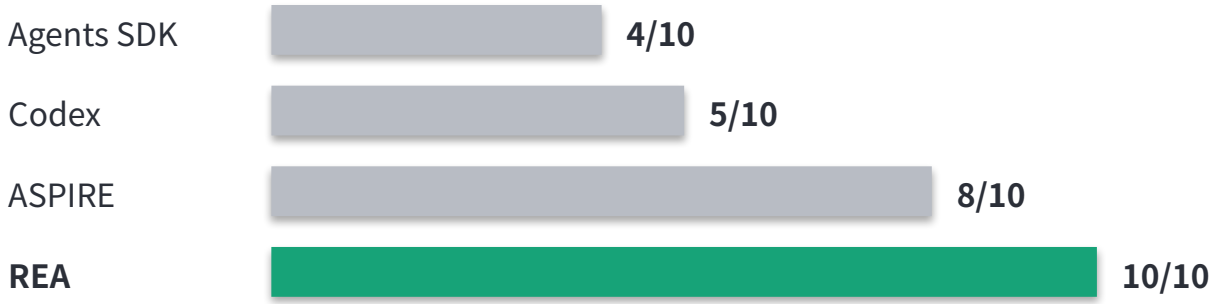
One runtime on hardware: G1 chair push 8/10, π0.5+code routing 8/10



Unitree G1 • approach and push a chair (operator-judged on site)



AlohaMini2 • put block in box (four methods, human-verified)



π0.5 alone 3/10 → REA (π0.5 + code) 8/10, placement failures 5 → 0; but groups collected on different days, and REA is slower.

Critical assessment

Uneven evidence: solid but small main gains, weaker extensions

01 Gain magnitude

Full set only +1.2 pp over best single method; oracle ceiling 96.9%. The success ceiling is still set by the brain-generated policy library (methods differ by 20–47%).

02 Scope of judge-gate evidence

Full-set main results use the exception gate; the judge's +15.3 pp comes from a hand-picked hard subset, only +3.1 pp on unselected seeds, at ~2.1× time.

03 VLA experiment controls

RoboTwin VLA baselines are historical runs from other time windows, MEDex on another host; one-shot selection at start only; code adds a net 2 successes.

04 Sample size and variance

BEHAVIOR: 50 ep, not significant vs SDK; real robot: 10 trials per method, human-judged; RoboTwin: same config differs by 36 scenes across nights.

05 Cold start

route memory is a per-task lookup needing dev episodes for each task; dispatch for unseen tasks is not discussed (except RoboTwin's learned selector).

06 Asynchrony coverage

In main LIBERO runs, the judge is called synchronously at boundaries, latency counted in budget; main tables omit async-advisor success; async numbers are in Fig. 3 notes and AlohaMini2.

Three takeaways

1

Use the brain beforehand and at boundaries, not in the running control loop

Generate policies offline + judge at policy return; in-loop consulting only adds waiting (blocking advisor idles 2.62 s/ep).

2

Routing on measured outcomes is cheap and effective

route memory, at 0 tokens, matches the 95.9% of online agent dispatch — and is the fastest.

3

Silent failures need boundary judging; the key is accurate done calls

done precision 0.83 / 0.905; pick the gate per library: free exception gate if policies rarely fail, judge if silent failures are common.